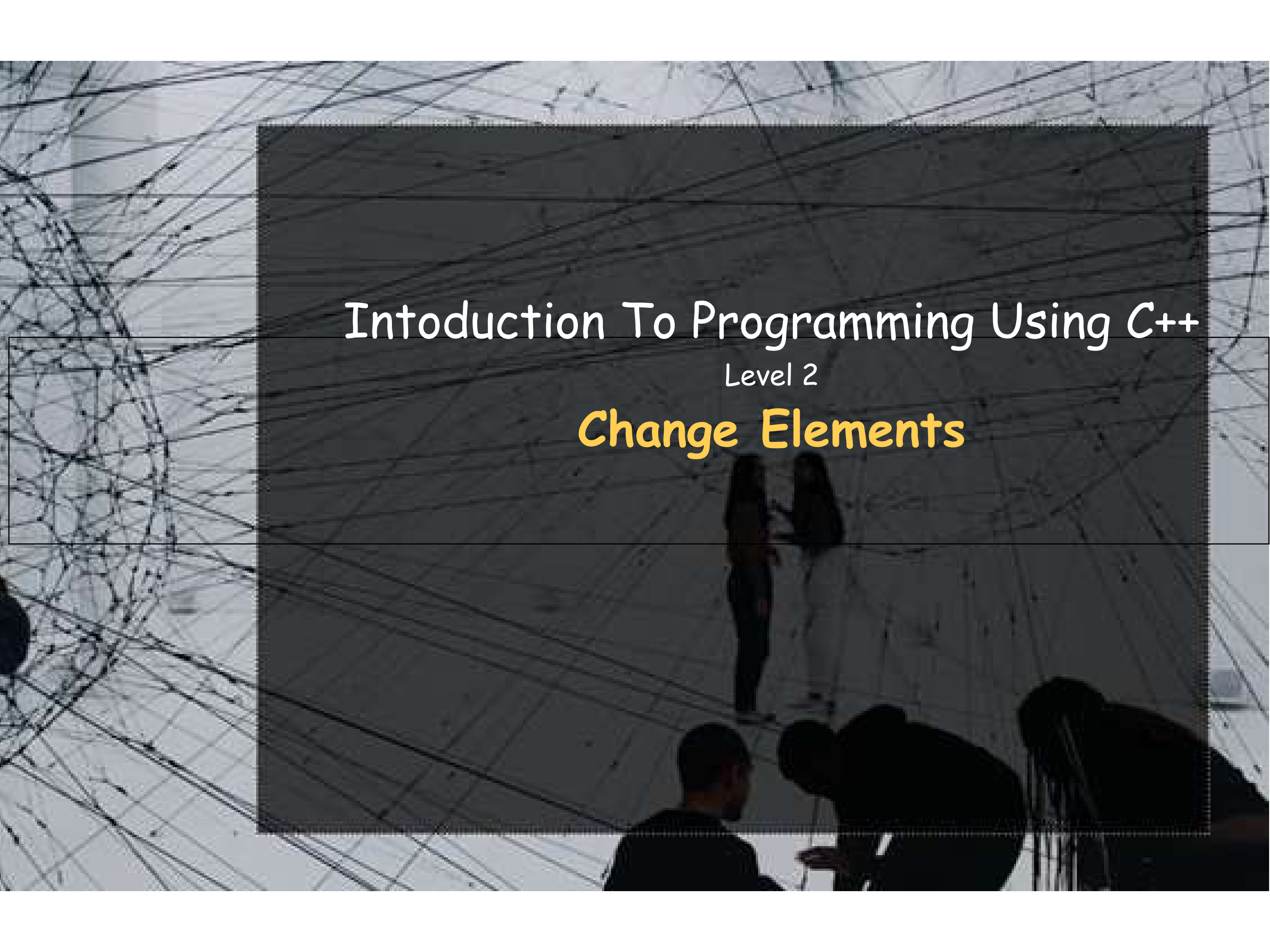




Introduction To Programming Using C++ Level 2

Change Elements



Introduction To Programming Using C++

Level 2

Change Elements

Definition

Changing Vector Elements

 Vector elements can be modified using a range-based `for` loop with a reference or by accessing a specific element using its index.

Explanation

- There are two common ways to change elements in a `vector` :
 - ▨ Using a reference (`&`) in a range-based `for` loop to modify every element.
 - ▨ Using an index or `.at()` to modify a specific element.

💡 Example

```
vector<int> num{1, 2, 3, 4, 5};  
// Change all elements to 20  
for (int& i : num)  
{  
    i = 20;  
}  
// Change specific elements  
num[1] = 40;  
num.at(2) = 80;  
num.at(4) = 90;
```

➡ Result

You can modify all vector elements using a range-based `for` loop with a reference.

You can modify specific elements using `index` or `.at()`.

Using `&` is important when you want changes to affect the original elements.

Note

- ◆ When using a reference (`int&`) inside a range-based `for` loop, you can modify the original elements directly inside the `vector`.
- ◆ If you use `int i` without `&`, `i` is only a copy, so changing it does not change the original vector element.

Comparison

`num[index]`

- ✓ Direct access to a specific element.
- ✓ Does not check whether the index is valid.
- ✓ Slightly less overhead.
- ✓ Use when you are sure the index is valid.

`num.at(index)`

- ✓ Accesses a specific element.
- ✓ Checks whether the index is within range.
- ✓ Throws an `out\_of\_range` exception if the index is invalid.
- ✓ Slightly more overhead.

✦ Summary

- ✓ Use range-based `for` with `&` to modify all elements.
- ✓ Use `index` or `.at()` to modify specific elements.
- ✓ `int& i` refers to the original vector element.
- ✓ `int i` creates a copy and does not modify the original element.

Thank you 😊